

Guardrail: Context-Driven Security Operations Platform

Project Workbook

By

Nikhil Dupally

Pooja Ramesh Sindham

Vijaya Sharavan Reddy Badddam

Viswa Surya Kumar Suvvada

March 4, 2026

Advisor: Prof. Park Younghee

Chapter 1. Literature Search, State of the Art

1.1 LITERATURE SEARCH

The problem of log anomaly detection is concerned with identifying unusual system behavior such as failures, misconfigurations, degradations in system performance, or unusual activity in systems and applications using log data. Log anomaly detection is a difficult problem since log data is available in huge volumes, is only semi-structured, and is subject to constant changes in software applications.

Early academic work formalized anomaly detection as a sequence learning problem. DeepLog models log data as sequences of events and uses LSTM-based predictions for identifying anomalies when sequences of events deviate from normal behavior[1]. However, advances in log anomaly detection critically depend on the availability of representative data for log sequences. To facilitate log anomaly detection using representative data, LogHub was created as a large collection of log data and a tool for reproducibility in log anomaly detection[2]. Log sparsity is a common problem in log data preprocessing. Drain is a popular online log parsing tool that is used for log sparsity reduction using a technique called clustering log messages into event templates[3].

More recent advances in log anomaly detection have focused on robustness and generalization in log anomaly detection. LogRobust is a system designed to make log anomaly detection more robust when dealing with unstable log data using a combination of semantic representations and sequence modeling[4]. LogRobust is less affected by changes in log event templates. LogAnomaly is a system for detecting both sequential and quantitative anomalies in unstructured log data. LogAnomaly is less affected by changes in log event templates using representations learned from log data[5]. LogBERT is a state-of-the-art log anomaly detection system that uses a transformer-based model for log anomaly detection using self-supervised learning objectives for normal log pattern learning and anomaly detection using BERT representations[6]. LogGPT is a state-of-the-art log anomaly detection system that uses a transformer-based model for log anomaly detection using log event representations trained using a variant of the GPT model. LogGPT also uses task-aligned model tuning for bridging the gap between log language modeling and log anomaly detection[7]. LLMeLog (ISSRE) is a state-of-the-art log anomaly detection system using LLM capabilities for deeper log event understanding for log anomaly detection[8].

1.2 RESEARCH PAPER INPUTS

The first step in our Guardrail strategy is to identify peer-reviewed and popularly used academic methods for dealing with anomalies in large-scale log data. Some of these include:

- DeepLog: Develops a baseline for detecting anomalies in log data using deep models for learning normal log event sequences.
- LogHub: Assists in reproducibility in log data analysis by providing a collection of publicly available log data and standardizing evaluation conditions.
- LogRobust: Focuses on log instability and provides a method for dealing with semantic representations.
- LogAnomaly: Enhances the ability to detect both sequence and quantitative anomalies in log data and reduces false positives due to template instability.
- LogBERT: Demonstrates the use of transformer models and self-supervised learning for log data.
- LogGPT: Demonstrates the use of a GPT/LLM model for log sequences and how the training objective can be adapted for log anomaly detection.
- LLMeLog: Demonstrates state-of-the-art log anomaly detection using LLMs and enhances our understanding of log events.

These works collectively shape Guardrail’s research grounding: sequence modeling baselines [1], reproducible benchmarking [2], scalable preprocessing foundations [3], robustness to evolving logs [4], improved anomaly sensitivity [5], transformer representations [6], and LLM-driven enhancements [7], [8].

1.3 STATE-OF-THE-ART SUMMARY

In the industry, the state-of-the-art in log anomaly detection is part of an observability stack that integrates well with metrics and traces, all designed for fast response to incidents, scalability in growth, and usability in day-to-day operations. Beyond the pure offline accuracy of the detection, the industry has been focusing on the following areas: (a) detection of spikes in log volumes and unusual patterns, (b) identification of suspicious fields/messages to aid in the investigation of the anomaly, (c) reduction of false positives through the incorporation of baselines and seasonality correction, and (d) integration of the results of the anomaly detection into the overall alerting stack.

For example, the Elastic Observability stack has a “Logs Anomalies” workflow that allows the user to spin up machine learning jobs for the analysis of log rate and anomaly detection directly from the user interface, designed to assist in operation as opposed to research-based evaluations. The industry has been moving towards semantic representations, context-based models such as the

“transformer” architecture, as well as the application of LLM-based analysis in the field of log anomaly detection to assist in the investigation of the anomaly results.

Guardrail targets this “leading-edge” industry direction by combining academically grounded anomaly detection methods [1]–[8] with practical design principles aligned to observability workflows like scalable preprocessing, sequence-based classification, evaluation reporting, and outputs that support investigation and benchmarking rather than only model training.

1.4 REFERENCES

- [1] M. Du, F. Li, G. Zheng, and V. Srikumar, “DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2017, pp. 1285–1298, doi: 10.1145/3133956.3134015.
- [2] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, “LogHub: A Large Collection of System Log Datasets for AI-driven Log Analytics,” *arXiv preprint*, arXiv:2008.06448, 2020.
- [3] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, “Drain: An Online Log Parsing Approach with Fixed Depth Tree,” in *Proc. IEEE Int. Conf. on Web Services (ICWS)*, 2017, doi: 10.1109/ICWS.2017.13.
- [4] X. Zhang *et al.*, “Robust Log-based Anomaly Detection on Unstable Log Data,” in *Proc. ACM Joint Meeting on European Software Engineering Conf. and Symp. on the Foundations of Software Engineering (ESEC/FSE)*, 2019, doi: 10.1145/3338906.3338931.
- [5] W. Meng *et al.*, “LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs,” in *Proc. IJCAI*, 2019.
- [6] H. Guo, S. Yuan, and X. Wu, “LogBERT: Log Anomaly Detection via BERT,” in *Proc. Int. Joint Conf. on Neural Networks (IJCNN)*, 2021, pp. 1–8, doi: 10.1109/IJCNN52387.2021.9534113.
- [7] X. Han, S. Yuan, and M. Trabelsi, “LogGPT: Log Anomaly Detection via GPT,” *arXiv preprint*, arXiv:2309.14482, 2023.
- [8] M. He *et al.*, “LLMeLog: An Approach for Anomaly Detection based on Large Language Model Enriched Log Events,” in *Proc. IEEE Int. Symp. on Software Reliability Engineering (ISSRE)*, 2024.
- [9] Elastic, “Inspect log anomalies (Logs Anomalies),” Elastic Observability Documentation.

Chapter 2. Project Justification

The justification for the Guardrail project is high, especially given that logs represent one of the first signs of failures, misconfigurations, security issues, or performance degradation in modern software systems. In production systems, logs are generated in large quantities in a constant stream. One of the key components of the response to these problems is the analysis of logs. Manually analyzing the data contained in the logs is not scalable. Simple alerting systems can have difficulty detecting new forms of failures or can have too many false positives. A system that can automatically identify abnormal behavior in the logs has clear advantages. These include the ability to decrease the time it takes for the system to detect the problems, the mean time to recovery (MTTR), and the overall reliability of the system.

The project is a substantial undertaking in software engineering and machine learning that spans two semesters. Guardrail is meant to provide a complete solution for log data preprocessing and normalization, sequence generation using windowing strategies, anomaly classification using machine learning models, and evaluation using standard metrics (Accuracy, Precision, Recall, and F1 score). The level of generality is suitable for both Part A and Part B because it covers both the essential design and implementation (data engineering, modeling, and evaluation) and the iterative process of improvement and evaluation (parameter tuning, robustness testing, and comparative testing). Moreover, the project is measurable because its performance can be evaluated using publicly available data and can be compared against other techniques in the literature as a baseline.

Guardrail has the potential to exceed expectations at the expo by providing results that can be published within the realm of applied software reliability engineering. A significant contribution to software reliability engineering can be achieved by including the following components: (a) the development of a reproducible anomaly detection workflow, (b) the evaluation of system performance using multiple benchmark datasets, and (c) the consideration of problem constraints such as robustness to changing log patterns, runtime considerations, and reporting. These components match the expectations of both industry and academic groups, as the work will be correct, reproducible, scalable, and operationally relevant. Guardrail can thus be positioned as a contribution to software reliability engineering, providing results that can be used to construct a peer-reviewed manuscript as well as a compelling expo demonstration.

Chapter 3. Project Requirements

3.1 PURPOSE AND OBJECTIVES

3.1.1 Purpose

The primary goal of Guardrail is to build a log-driven anomaly detection system, which examines the log entries of a system and its applications, and then labels them as Normal or Anomalous to detect potential failures and misconfigurations in software systems.

3.1.2 Objectives

The project objectives for building the log anomaly detection workflow include:

- a. Developing a complete workflow for data preparation, model execution, and evaluation. (data preparation → model execution → evaluation).
- b. Categorizing the log sequences into anomalous (unusual) behavior and normal behaviour.
- c. Providing the ability for log preprocessing, where variable tokens such as IDs, timestamps, and addresses can be masked/normalized.
- d. Creating labeled sequences of logs by applying windowing techniques such as sliding windowing and session windowing.
- e. Displaying evaluation results, including Accuracy, Precision, Recall, and F1-score.
- f. Providing the ability for the model to be loaded for testing.
- g. Providing the ability for the workflow to be executed without any manual intervention during execution, based on the provided configuration.

3.2 USE CASES / SCRUM USER STORIES

US-1: As a user, I want the log dataset path to be provided to Guardrail to allow the ingestion of the logs for further analysis.

US-2: As a user, I want the logs to be normalized and masked to prevent the tokens of variables from impacting the robustness of the model.

US-3: As a user, I want Guardrail to produce sequences by using windowing to allow the detection of behavioral patterns in the temporal dimension.

US-4: As a user, I want Guardrail to classify the sequences as Normal or Anomalous to allow for the automatic detection of abnormal behavior.

US-5: As a user, I want Guardrail to load a model checkpoint to allow for the continuation of the inference without the need to retrain the model.

US-6: As a user, I want Guardrail to produce evaluation metrics to allow for the reporting of the results.

US-7: As a user, I want Guardrail to produce explicit error messages when the input is missing or invalid to allow for the prompt resolution of the problem.

3.3 SCOPE OF WORK

Following is the scope for the project:

- a. The scope of guardrail would be to support anomaly detection over at least one publicly available log data set (e.g., HDFS, BGL, Liberty, Thunderbird).
- b. The scope would also include implementation and validation of:
 - i. Log preprocessing and normalization
 - ii. Sequence generation for training and testing
 - iii. Model inference and evaluation
- c. The classification model would be based on deep learning techniques.
- d. The log sequence would be used in the classification model instead of log statements.
- e. The predictions would be generated to be used in the final project report.

3.4 NON-FUNCTIONAL REQUIREMENTS

In line with the specified purpose and objectives, the following non-functional requirements can be identified:

- a. Reproducibility: The system should be able to produce the same metrics when provided with the same set of data, checkpoint, and configuration, within a specified tolerance level.
- b. Performance/stability: The system should be able to perform batch processing, including the specification of sequence limits to prevent memory failures.

- c. Configurability: The system should allow the user to configure the path of the dataset, window size, batch size, and checkpoint path without requiring code modifications.
- d. Robustness to variability: The system should be able to reduce the impact of variable tokens to enhance robustness to log variability.
- e. Maintainability: The system should be able to perform the following as separate components with well-defined inputs and outputs: preprocessing, sequence generation, model execution, and evaluation.
- f. Usability: The system should be able to show runtime status messages to facilitate easier debugging and reporting.

3.5 FUNCTIONAL REQUIREMENTS

3.5.1 Essential Features

- a. The system shall ingest log data from chosen datasets and formats through a specified input path.
- b. The system shall preprocess log data using normalization and masking rules and shall provide preprocessed log data or log sequences.
- c. The system shall generate labeled log sequences for learning and evaluation using a sliding window method and/or a session windowing method.
- d. The system shall conduct binary classification on log sequences using a Normal/Anomalous model and shall produce a predicted label for each log sequence.
- e. The system shall allow for an existing checkpoint to be loaded and test data to be processed for inference without retraining.
- f. The system shall calculate evaluation metrics for test data (Accuracy, Precision, Recall, F1 score).
- g. The system shall export results in a reusable form:
 - i. a summary of metrics (console output and/or file);
 - ii. prediction results for each log sequence (file or equivalent).
- h. The system shall validate critical inputs (dataset path and checkpoint path) and terminate execution with a clear error message in case of failure.

3.5.2 Desired Features

- a. The system shall provide for training or fine-tuning operations, which shall be dependent on available resources, for generating a usable model checkpoint.
- b. The system shall provide for execution with a fixed random seed configuration.
- c. The system shall provide for explicit configurability of parameters without requiring changes to code.
- d. The system shall provide for basic input validation, including proper error messages for missing paths, formats, and parameters.

3.5.3 Optional Features

- a. The system may include support for evaluation across several datasets to enable comparison.
- b. The system may include support for generating summary reports automatically, which can include metrics tables and simple plots.
- c. The system may include support for generating limited interpretability results, which can include highlighting log lines or tokens for debugging.

Chapter 4. Dependencies and Deliverables

4.1 Project Deliverables

The project deliverables are as follows for the Guardrail project:

4.1.1 Technical Deliverables

- a. A functional Guardrail anomaly detection pipeline with components for preprocessing, sequence generation, inference, and evaluation.
- b. A functional module for data preprocessing and windowing that transforms raw data into model-ready sequences, with the processed data sets stored as dataset artifacts.
- c. A functional inference workflow that can load the trained model checkpoint and generate the predictions on the given test dataset.
- d. A functional evaluation workflow that can run and provide Accuracy, Precision, Recall, and F1-score for the given dataset.
- e. Model artifact used for the final evaluation (a trained checkpoint developed by the team or a validated checkpoint commonly used for the results).
- f. A results package with the prediction results and the final metrics summary table for inclusion in the final report.
- g. Architectural documentation on the system diagram with a brief data flow and model flow specific for the Guardrail project.

4.2 Dependencies

Following are the list of potential dependencies that could cause a project halt if they are not met:

4.2.1 Data Dependencies

- a. Availability of at least one benchmark log dataset in an accessible form (i.e., raw logs or preprocessed files) with an appropriate train/test or evaluation split.
- b. Availability of ground-truth labels (or an established labeling scheme) required to evaluate anomaly detection performance.
- c. Capability to generate sequences (i.e., windowing) without data corruption or leakage (i.e., logs must have appropriate fields and order).

4.2.2 Compute / Runtime Dependencies

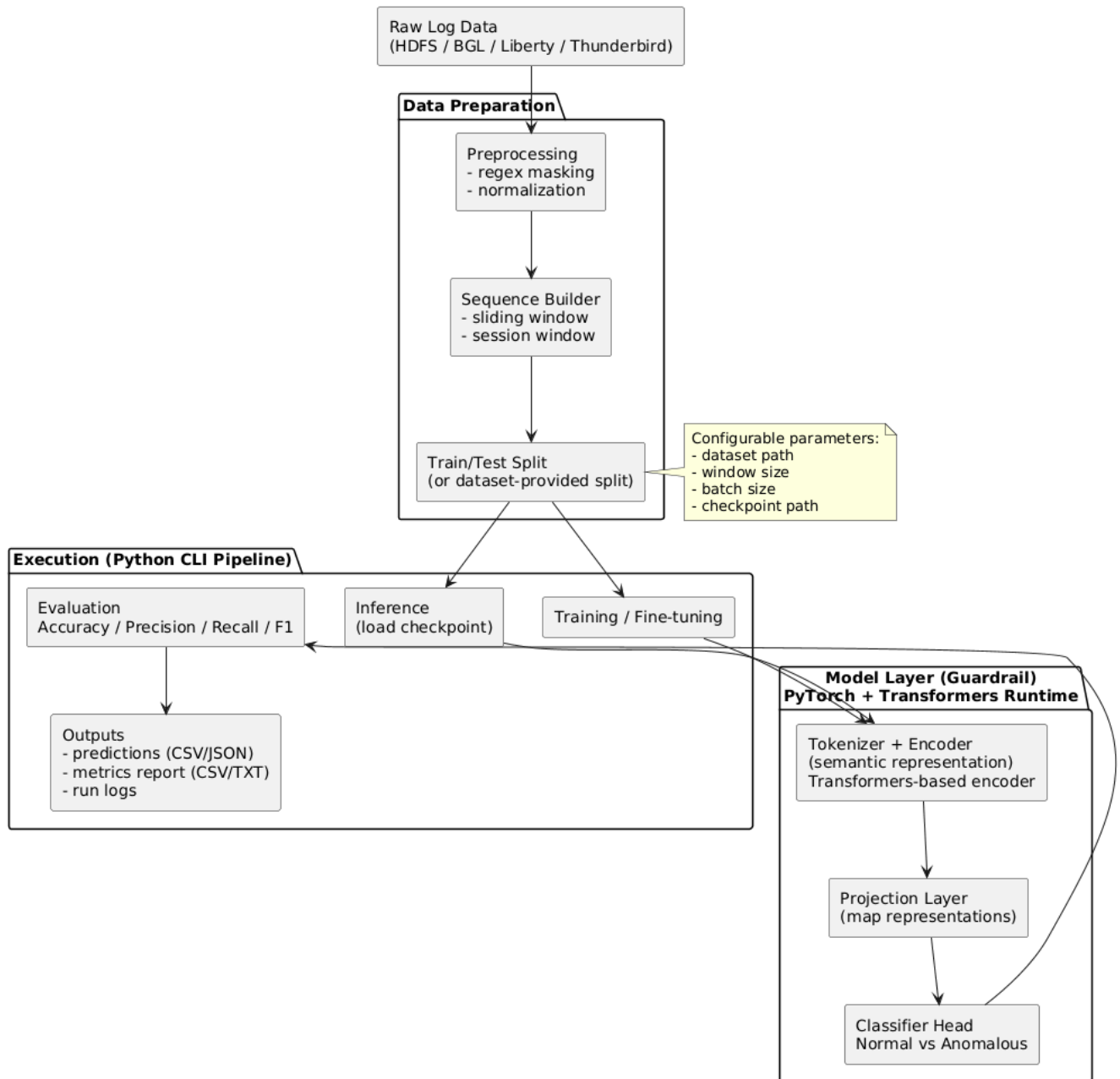
- a. Availability of a working execution environment that can run the model pipeline (i.e., at least CPU if used for small-scale evaluation; GPU if training or fine-tuning is performed).
- b. Sufficient storage to hold dataset files, sequences, and checkpoint data.

4.2.3 Software Compatibility Dependencies

- a. Availability of a working Python environment on which required ML software libraries can be installed and executed.
- b. Compatibility of software library versions required to load/run checkpoints and execute inference/evaluation scripts.

Chapter 5. Project Architecture

Guardrail - Log Anomaly Detection Architecture



The Guardrail architecture has been designed to be a layered architecture for log anomaly detection in batch logs. The architecture reads raw log data (e.g., HDFS, BGL, Liberty, Thunderbird), passes it through a data preparation stage to normalize and mask noise in log data due to varying tokens like timestamps, identifiers, and addresses. The log sequence builder then uses either a sliding window or session window to build log sequence data. The log sequence data is then divided into training and testing datasets or uses the provided splits in the datasets.

The Execution Layer has two workflows: Inference/Evaluation and optional Training/Fine-tuning. In Inference/Evaluation, the trained checkpoint is used to execute sequence classification. In Training/Fine-tuning (optional), training sequences are consumed to generate a checkpoint for Evaluation.

The Model Layer has three components: Tokenizer/Encoder, Projection Layer, and Classifier Head. The Tokenizer/Encoder uses log sequence data to generate semantic representations using a log sequence tokenizer/encoder. The Projection Layer then uses log sequence data to generate log sequence projection using log sequence projection. The Classifier Head then uses log sequence projection to generate binary class label output.

The output of the Guardrail architecture consists of a results package containing sequence level predictions and evaluation metrics like Accuracy, Precision, Recall, and F1-score.

The architecture has been designed to be modular with a layered architecture (data preparation, model layer, execution layer), which is in line with AI/SE architecture styles like Pipe and Filter architecture.

Chapter 6. Project Design

[

Describe your project design. Types of design artifacts that you provide might include:

- UML diagrams (class diagrams, sequence diagrams, etc.)
- UI Mockups
- Database Entity diagrams
- Hardware block diagrams
- Other artifacts you deem necessary to describe your project in detail.
- CMPE Students may want to consider the following alternatives:
 - Hardware Block Diagram
 - System Block Diagram (Detailed block diagram for each block)
 - Circuit Diagram (as applicable)

Evaluated: Workbook Assignment 2

]

Chapter 7. QA, Performance, Deployment Plan

[

Describe the testing, performance evaluation, and deployment plan for your project

CMPE students may want to consider the following alternatives

- Testing: Different tests done or to be done on the project and their outcomes
- Algorithms
- Bill of materials

·

· Evaluated: Workbook Assignment 2]

Chapter 8. Implementation Plan and Progress

This chapter outlines the overall plan for the implementation of Guardrail and a progress tracker that is intended to be updated continuously throughout the project. Guardrail is intended to be a reproducible, end-to-end log anomaly detection system that takes raw log data and converts it into a format that is suitable for model inputs, performs anomaly classification, and produces evaluation outputs that are suitable for benchmarking and reporting. It is divided into phases that reflect how the project will be built from setup through final completion.

8.1 Implementation Plan (Project-wide)

Phase 1: Project Initialization and Execution Readiness

Goal: Make Guardrail runnable in a consistent way across the team.

Activities:

- Create a standardized project run structure (input folder, output folder, configuration file/parameters).
- Establish a consistent Python setup for all team members and confirm required libraries install successfully.
- Confirm the team can run a minimal command/script without dependency errors.

Completion Criteria:

- A documented setup procedure exists, and at least one baseline script runs successfully end-to-end on a small input.

Phase 2: Data Flow Validation (Logs → Sequences)

Goal: Validate that raw logs can be converted into high-quality model inputs.

Activities:

- Create and validate rules for data preparation (masking and normalization) to prevent certain tokens from dominating the data representation.
- Create and validate the creation of sequences using windowing methods (sliding and/or sessions) and verify that the order and labels of the data are preserved.
- Verify adherence to train-test separation (no leakage) during sequence creation.

Completion Criteria:

- The creation of model-ready data sets (train and test sequences) in a consistent form, repeated.

Phase 3: Model Execution Baseline (Inference + Metrics)

Goal: Validate that Guardrail can run inference and produce measurable evaluation results.

Activities:

- Use a checkpoint to do inference and produce Normal and Anomalous predictions for all sequences.
- Evaluate and calculate Accuracy, Precision, Recall, and F1-score.
- Save the results in a reusable format, such as prediction, evaluation, and log output.

Completion Criteria:

- A reproducible baseline run exists for at least one dataset with saved prediction files and a metrics summary.

Phase 4: Training/Fine-tuning and Checkpoint Production (Resource Permitting)

Goal: Produce a team-generated model checkpoint and evaluate improvements.

Activities:

- Configure and run training/fine-tuning using controlled parameters (epochs, batch size, seed).
- Save a new checkpoint and evaluate using the same evaluation workflow as the baseline.
- Compare trained results with baseline and document differences.

Completion Criteria:

- At least one team-generated checkpoint is produced and evaluated, or training is formally scoped out with documented constraints and a checkpoint-based evaluation plan.

Phase 5: Benchmarking and Robustness Evaluation

Goal: Strengthen the project contribution by validating behavior across settings and datasets.

Activities:

- Run benchmarking experiments (different window sizes, masking strategies, batch sizes, and seeds).
- If feasible, run evaluation across more than one dataset to assess generalization.
- Produce a consolidated benchmarking table and short interpretation of results.

Completion Criteria:

- Benchmark results are consolidated into tables with clear comparisons and reproducible run settings.

Phase 6: Reliability Hardening and Final MVP Packaging

Goal: Make Guardrail stable, reproducible, and ready for final submission/demo.

Activities:

- Improve reliability (fail-fast validation, clear error messages, structured logging).
- Improve stability/performance (batching, memory-safe sequence limits, consistent output generation).
- Package the final “reproduce this run” workflow: configuration, scripts, checkpoint, and output structure.

Completion Criteria:

- A new user can follow documented steps to execute Guardrail end-to-end and reproduce the final reported metrics.

8.2 Progress Tracker (to be updated continuously)

Phase	Current Status	Evidence to Record

Phase 1: Project initialization and execution readiness	In progress	Setup notes, dependency install confirmation, first successful run log
Phase 2: Data flow validation (logs → sequences)	Planned	Sample processed output files, sequence samples, leakage checks
Phase 3: Model execution baseline (inference + metrics)	Planned	Prediction file, metrics table, evaluation log
Phase 4: Training/fine-tuning and checkpoint production	Planned	Training logs, saved checkpoint, evaluation comparison
Phase 5: Benchmarking and robustness evaluation	Planned	Benchmark tables, configuration variants, summary analysis
Phase 6: Reliability hardening and final MVP packaging	Planned	Final run instructions, reproducibility proof, demo-ready results

8.3 Current Progress Summary

- Guardrail documentation has been prepared for core workbook chapters (justification, requirements, dependencies/deliverables, architecture).
- The project code package contains core components required for implementation: data preparation (preprocessing and windowing), model execution (training/inference), and evaluation workflows, along with model artifacts suitable for validation runs.
- The next measurable implementation steps are completing environment readiness, validating data preparation outputs, and producing baseline inference + evaluation metrics on at least one dataset.

Chapter 9. Project Schedule

SPRING 2026 (Part A)

Goal: Foundation + feasibility POCs + baseline run + WB2 hardening + Part A consolidation

Task No.	Task Description	Team Member	Tentative Start Date	Tentative End Date	Milestone/POC
M1	Phase 1 — Project Initialization & Execution Readiness	Team	Mar 4, 2026	Mar 10, 2026	Milestone 1: Environment + Repo Ready
1.1	Create repo structure (inputs/outputs/configs/logs), branching conventions	Nikhil Dupally	Mar 4, 2026	Mar 5, 2026	M1
1.2	Set up Python environment + dependency install guide (venv/conda)	Pooja Ramesh Sindham	Mar 4, 2026	Mar 6, 2026	M1
1.3	Validate baseline script execution (imports/run smoke test), capture run logs	Vijaya Sharavan Reddy Badddam	Mar 4, 2026	Mar 6, 2026	M1
1.4	Define configuration template (paths, window size, batch size, checkpoint path)	Viswa Surya Kumar Suvvada	Mar 4, 2026	Mar 6, 2026	M1
M2	Phase 2 — Data Flow Validation (Logs → Sequences)	Team	Mar 11, 2026	Mar 31, 2026	Milestone 2: Data Pipeline Feasibility
2.1	Implement/validate preprocessing rules (masking/normalization) + sample outputs	Nikhil Dupally	Mar 11, 2026	Mar 17, 2026	POC-1: Preprocessing
2.2	Implement/validate sliding window sequence generation + format checks	Pooja Ramesh Sindham	Mar 11, 2026	Mar 20, 2026	POC-2: Sliding Window

2.3	Implement/validate session window sequence generation (dataset dependent)	Vijaya Sharavan Reddy Badddam	Mar 18, 2026	Mar 26, 2026	POC-3: Session Window
2.4	Validate train/test split logic + leakage checks + final dataset artifact layout	Viswa Surya Kumar Suvvada	Mar 21, 2026	Mar 31, 2026	POC-4: Split Validation
M3	Phase 3 — Baseline Model Execution (Inference + Metrics)	Team	Apr 1, 2026	Apr 6, 2026	Milestone 3: Baseline Run
3.1	Select dataset + prepare final run configuration for baseline	Nikhil Dupally	Apr 1, 2026	Apr 2, 2026	M3
3.2	Run checkpoint inference and generate predictions (store outputs)	Pooja Ramesh Sindham	Apr 2, 2026	Apr 4, 2026	M3
3.3	Run evaluation metrics (Accuracy/Precision/Recall/F1) and save metrics table	Vijaya Sharavan Reddy Badddam	Apr 3, 2026	Apr 5, 2026	M3
3.4	Package run artifacts (predictions, metrics, logs, config snapshot)	Viswa Surya Kumar Suvvada	Apr 5, 2026	Apr 6, 2026	M3
M4	Phase 4 — Pipeline Hardening & Repeatability (WB2 Focus)	Team	Apr 7, 2026	May 1, 2026	Milestone 4: Repeatable Pipeline
4.1	Add input validation + fail-fast error messages (missing paths/checkpoints)	Nikhil Dupally	Apr 7, 2026	Apr 15, 2026	M4
4.2	Standardize output formats + folder structure (metrics + predictions + logs)	Pooja Ramesh Sindham	Apr 7, 2026	Apr 15, 2026	M4
4.3	Run repeatability tests (2 configs or 2 runs) + collect comparison metrics	Vijaya Sharavan Reddy Badddam	Apr 16, 2026	Apr 25, 2026	M4

4.4	Build benchmarking table + short interpretation write-up	Viswa Surya Kumar Suvvada	Apr 26, 2026	May 1, 2026	M4
M5	Phase 5 — Part A Consolidation & Handoff Plan	Team	May 2, 2026	May 15, 2026	Milestone 5: Part A Complete
5.1	Consolidate reproducible run instructions (“reproduce this run” guide)	Nikhil Dupally	May 2, 2026	May 6, 2026	M5
5.2	Clean configs + finalize dataset artifact versioning/naming conventions	Pooja Ramesh Sindham	May 2, 2026	May 6, 2026	M5
5.3	Document limitations + risk log (compute/data/accuracy constraints)	Vijaya Sharavan Reddy Badddam	May 7, 2026	May 11, 2026	M5
5.4	Define Fall roadmap (training, benchmarking, optimization, MVP packaging)	Viswa Surya Kumar Suvvada	May 12, 2026	May 15, 2026	M5

FALL 2026 (Part B)

Goal: Training (if feasible) + multi-dataset benchmarking + optimization + final MVP/demo package

Task No.	Task Description	Team Member	Tentative Start Date	Tentative End Date	Milestone/POC
M6	Phase 6 — Training/Fine-tuning & Checkpoint Production	Team	Aug 20	Sep 20	Milestone 6: Team Checkpoint Ready

6.1	Configure training parameters + reproducible seed plan + run template	Nikhil Dupally	Aug 20	Aug 30	M6
6.2	Prepare training-ready dataset artifacts + verify compatibility with training	Pooja Ramesh Sindham	Aug 20	Aug 30	M6
6.3	Execute training/fine-tuning run(s) and store checkpoint versions	Vijaya Sharavan Reddy Badddam	Aug 31	Sep 12	M6
6.4	Evaluate new checkpoint vs baseline + summarize delta	Viswa Surya Kumar Suvvada	Sep 13	Sep 20	M6
M7	Phase 7 — Benchmarking & Robustness Evaluation	Team	Sep 21	Oct 25	Milestone 7: Benchmark Tables Complete
7.1	Robustness experiment set A (window size sensitivity)	Nikhil Dupally	Sep 21	Oct 05	M7
7.2	Robustness experiment set B (masking strategy variants)	Pooja Ramesh Sindham	Sep 21	Oct 05	M7
7.3	Multi-dataset evaluation run(s) (if feasible) + results capture	Vijaya Sharavan Reddy Badddam	Oct 06	Oct 18	M7
7.4	Consolidated benchmarking tables + analysis summary	Viswa Surya Kumar Suvvada	Oct 19	Oct 25	M7
M8	Phase 8 — Performance Optimization & Stability Hardening	Team	Oct 26	Nov 20	Milestone 8: Stable MVP Build
8.1	Memory safety improvements (batching, sequence limits, guardrails)	Nikhil Dupally	Oct 26	Nov 06	M8

8.2	Improve logging + run metadata capture for reproducibility	Pooja Ramesh Sindham	Oct 26	Nov 06	M8
8.3	Regression testing across configs + failure-mode checks	Vijaya Sharavan Reddy Badddam	Nov 07	Nov 15	M8
8.4	Final performance report + readiness checklist	Viswa Surya Kumar Suvvada	Nov 16	Nov 20	M8
M9	Phase 9 — Final MVP Packaging, Documentation, and Demo	Team	Nov 21	Dec 15	Milestone 9: Final Deliverable
9.1	Final “one-command run” packaging (config + outputs + checkpoint pointers)	Nikhil Dupally	Nov 21	Nov 30	M9
9.2	Final documentation bundle (architecture + how-to-run + experiment summary)	Pooja Ramesh Sindham	Nov 21	Nov 30	M9
9.3	Final end-to-end demo run + metrics verification + regression sign-off	Vijaya Sharavan Reddy Badddam	Dec 01	Dec 08	M9
9.4	Demo narrative + final presentation material + submission checklist	Viswa Surya Kumar Suvvada	Dec 09	Dec 15	M9